GSVmulti und Raspberry Pi

Eigene Sensoren über USB und Netzwerk anbinden

Technische Anleitung für GSVmulti 2.01 Pro unter Windows
Recherche und Dokumentation vom 28. Oktober 2026

Ergebnis

Dein Raspberry Pi kann für GSVmulti PRO als benutzerdefinierte Messwertquelle auftreten. Er liest den eigentlichen Sensor aus und liefert daraus einen regelmäßigen Textdatenstrom. In GSVmulti wird dafür **Device Type = UserDefined** verwendet. Ein vollständiger Nachbau eines GSV-2-, GSV-6- oder GSV-8-Geräteprotokolls ist für diesen Weg nicht erforderlich. [1]

Für einen generischen IP-Datenstrom ist in deiner Version 2.01 Pro der hier belegte Weg eine **Abbildung von TCP auf einen Windows-COM-Port**. Im gezeigten Add-Channel-Dialog wird eine COM-Nummer gewählt; ein allgemeiner TCP-Host mit Port ist dort nicht vorgesehen. Die Herstellerlösung für serielle Netzwerkgeräte arbeitet ebenfalls mit COM-Mapping. [2]

Ziel	Empfohlener Aufbau	Zusätzliche TCP-COM-Brücke
Einfacher lokaler Anschluss	Pi UART -> 3,3-V-USB-UART am Windows-PC	Nein
Lokaler Anschluss ohne GPIO	Pi USB-UART -> gekreuzte UART-Leitungen -> PC USB-UART	Nein
LAN oder WLAN	Pi TCP-Server -> Windows-VCOM -> GSVmulti	Ja
Ein USB-Kabel am geeigneten Pi	USB-ACM-Gadget -> Windows-COM	Nein; USB-Seriell-Treiber erforderlich

Was die Anleitung abdeckt

Geräte und Kanäle, Textformat, Portauswahl, UART-Verdrahtung, USB-Gadget als Alternative, TCP-Weiterleitung, Grenzen der Zeitbasis sowie eine schrittweise Abnahme. Das beiliegende Beispielpaket erzeugt drei Demokanäle und enthält eine optionale Windows-Bridge. Die realen Sensortreiber musst du passend zu deinem Sensor ergänzen.

Stand der Überprüfung

Bestätigt sind die lokale Installation unter C:\Program Files\GSV-Multichannel v2.01 und der laufende Prozess; die Anhänge zeigen 2.01 Pro und UserDefined. Die Detaildefinition stammt aus der aktuellen Herstelleranleitung v2.6, Seiten 118 bis 121. Die Live-Oberfläche ließ sich beim Prüfen nicht zuverlässig aktivieren. Ein Empfangstest in GSVmulti, ein Raspberry-Pi-Hardwaretest und ein installierter VCOM-Treiber wurden daher **nicht** bestätigt. Konkrete Aufbauten und Beispiele sind nachfolgend als Umsetzungsvorschlag beschrieben.

1 Wie Sensor Gerät und Kanal zusammenhängen

Ein Sensor erzeugt zunächst eine physikalische oder digitale Messinformation. Der Raspberry Pi muss diese Information auslesen, gegebenenfalls digitalisieren, kalibrieren und in eine Zahl umrechnen. Erst danach beginnt die Verbindung zu GSVmulti.

```
Sensor -> ADC / I2C / SPI / UART / USB -> Pi-Messprogramm
       -> Messwerttext -> COM-Transport -> GSVmulti-Kanaele
       -> Live-Anzeige -> Aufzeichnung -> Datei / Export
```

Begriff	Bedeutung im vorgeschlagenen Aufbau
Sensor	Die tatsächliche Messquelle, etwa ein Wegsensor, ADC oder digitales Messgerät.
Device	Eine Datenquelle mit gemeinsamer Verbindung und gemeinsamem Frameformat, hier das Pi-Programm.
Input Channel / Position	Die Position einer Zahl innerhalb eines Messwertframes. Position 2 bedeutet den zweiten Zahlenwert.
GSVmulti-Kanal	Der angezeigte und aufgezeichnete Messwert, der einer solchen Position zugeordnet wird.
COM-Port	Der von Windows angebotene serielle Endpunkt. Kann von Hardware, USB oder einem virtuellen Treiber stammen.

Ein Port kann mehrere Messgrößen liefern

Für drei Sensorwerte benötigst du in diesem Entwurf einen Port mit drei Zahlen pro Zeile. Jeder Datensatz enthält dieselben drei Größen in derselben Reihenfolge. Die Sensoren werden auf dem Pi zusammengeführt. Es sind keine drei USB-Kabel erforderlich.

Eine Kanalposition hat keine Beziehung zu einer Raspberry-Pi-GPIO-Nummer. GSVmulti weiß beispielsweise nicht, dass der zweite Wert ursprünglich über I2C eingelesen wurde. Diese Zuordnung hältst du in der Dokumentation deines Pi-Programms fest.

Native GSV-Geräte und eigene Messwertquellen

Bei einem nativen GSV-Gerät übernimmt GSVmulti auch gerätespezifische Konfiguration. Die unterstützten Messdatenframes können binär oder gerätespezifisch als Text vorliegen. Die Einstellung Device -> Advanced Settings -> Value Mode gehört zu diesem Gerätepfad. Sie definiert keinen beliebigen Parser für einen fremden Raspberry Pi. [3]

Bei UserDefined ist dein Pi dagegen der Lieferant fertiger Zahlen. Sensorkalibrierung, ADC-Einstellungen und Sensorbus-Kommandos gehören in dein Messprogramm. „Special Sensor“ im Hauptfenster ist für sich allein kein Nachweis einer frei programmierbaren Sensortreiber-Schnittstelle.

TDMS sowie Text-/Excel-Export betreffen die Speicherung und Auswertung. Sie sind von dem seriellen Live-Eingangsformat zu unterscheiden. Eine CSV-Datei ist kein automatisch geöffneter Live-Kanal. [4]

2 Das akzeptierte Textformat

Die folgende kompakte Referenz fasst die vom Hersteller beschriebene UserDefined-Schnittstelle zusammen. Sie ist die Grundlage für die eigenen Beispielprofile. [1, Seiten 118 bis 121]

Eigenschaft	Anforderung beziehungsweise Auswahl
Verbindung	Echte oder virtuelle serielle Schnittstelle; 8 Datenbits, keine Parität, 1 Stopbit (8N1).
Bitraten	2400, 4800, 9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600.
Daten	ASCII beziehungsweise UTF-8; selbstständiges Senden ohne Messwertabfrage; gleichbleibender Aufbau; Dezimalpunkt.
Frameende	CR LF, Bytes 0D 0A. In Python: \r\n. Die Kürzel sind an einer Stelle der Anleitung vertauscht.
Frame Prefix Mode	Has Character Prefix; Has Binary Prefix; No Prefix, 1st char is - or digit.
Präfix	Ein Zeichen oder ein Byte als zweistellige Hexzahl; ohne Präfix beginnt die Zeile mit Vorzeichen oder Ziffer.
Number of values in String/Frame	Tatsächliche Zahl der Werte im Frame, auch wenn nur ein Teil davon angezeigt wird.
Value Separator	Komma, Semikolon, Pipe, Leerzeichen, Tab oder ein eigenes Zeichen.
Filter-Out Non-Numeric	Kann Begleittext herausfiltern. Für den hier vorgeschlagenen reinen Zahlenstrom ausgeschaltet lassen.

Empfohlenes eigenes Profil

```
%+00012.345;+00023.456;-00000.250\r\n
```

Das Prozentzeichen synchronisiert den Framebeginn. Danach folgen drei Zahlen mit Punkt und drei Nachkommastellen, getrennt durch Semikolon. Die letzten beiden Bytes sind die Steuerzeichen CR und LF. Die im Beispiel sichtbare Schreibweise \r\n steht für diese zwei Bytes; es werden keine vier Backslash-Buchstaben gesendet.

Diese konkrete feste Feldbreite ist eine eigene Designentscheidung für leichte Sichtprüfung. Entscheidend für den Parser ist der gleichbleibende Aufbau. Aus der Herstellerforderung folgt nicht, dass jeder Zahlenwert zwingend dieselbe Anzahl Ziffern haben muss.

Für den ersten Versuch keine Einheiten, Feldnamen, JSON, Zeitstempel oder Debug-Ausgaben mitsenden. So bleiben Formatfehler von Verdrahtungs- und Portproblemen unterscheidbar.

Eine maximale UserDefined-Kanalzahl oder Framegröße sowie die Behandlung wissenschaftlicher Zahlenschreibweise sind in den ausgewerteten Angaben nicht verbindlich spezifiziert. Das Beispiel nutzt bewusst drei Kanäle, kurze ASCII-Zeilen und gewöhnliche Dezimalzahlen.

3 Beispiele und Zuordnung der Messwerte

Profil A für einen einzelnen Sensorwert

```
+12.345\r\n
```

Umsetzungsvorschlag: kein Präfix, Number of values = 1. Der Wert wird Position 1 zugewiesen. Zum Einstieg ist auch dieses Minimalformat geeignet; das Beispielprogramm im Paket verwendet jedoch durchgehend das dreikanalige Profil B.

Profil B für das beiliegende Beispielprogramm

Feld im DEFINE-Dialog	Einstellung des Beispielprofils
Frame Prefix Mode	Has Character Prefix
Define Prefix Character	%
Number of values in String/Frame	3
Value Separator	Semicolon ;
Filter-Out Non-Numeric	Aus

Position	Wert aus der Beispielzeile	Vorgesehene Zuordnung
1	+00012.345	Messgröße A; Einheit separat festlegen
2	+00023.456	Messgröße B; Einheit separat festlegen
3	-00000.250	Messgröße C; Einheit separat festlegen

Das Prozentzeichen und das Frameende sind keine Messwerte. Ein Datensatz steht hier für einen gemeinsamen Abtastschritt. Wenn die Sensoren nacheinander gelesen werden, muss dein Pi-Programm die daraus entstehende zeitliche Differenz berücksichtigen.

Was dieses Profil nicht akzeptiert

Beispiel	Problem
%12,345;23,456;-0,250	Komma als Dezimalzeichen statt Punkt.
%12.345;23.456	Nur zwei Werte, obwohl drei konfiguriert sind.
{"force":12.345}	JSON ist nicht das hier konfigurierte Zahlenprofil.
sensor ready	Statusmeldung im Messdatenstrom.
%12.345;23.456;-0.250 + nur LF	Unvollständiger Zeilenabschluss für dieses Profil.

Ein ganz anderer Textaufbau kann mit eigener Definition eventuell funktionieren. Für JSON, Binärfloats, beliebige CSV-Regeln mit Anführungszeichen oder freie Statusmeldungen ist durch die Dokumentation kein universeller Import zugesichert. Dafür einen Adapter auf dem Pi oder vor dem COM-Port verwenden.

4 Den Kanal in GSVmulti konfigurieren

Die folgenden Schritte beziehen sich auf den Add-Channel-Dialog in deinem Screenshot. Die Werte % und 3 gehören zum beiliegenden Beispielsender. Der tatsächliche Port muss vorher unter Windows existieren.

1. Den seriellen Datenweg oder die Netzwerk-COM-Brücke vollständig einrichten. In Windows den resultierenden Port prüfen. COM20 ist nachfolgend ein frei wählbares Beispiel; COM122 aus deinem Screenshot ist kein Nachweis eines angeschlossenen Geräts.

2. Add Channel öffnen. Unter Device Type die Auswahl **UserDefined** treffen. Voraussetzung ist die PRO-Funktion für benutzerdefinierten Messwerttext. [1]

3. Value String / DEFINE öffnen und das Profil von Seite 4 dieser Dokumentation einstellen: Zeichenpräfix %, drei Zahlen, Semikolon, Textfilter aus.

4. COMport Number auf den wirklichen Windows-Port setzen. Wenn der Port nicht in der Auswahl erscheint, Portinstallation und Anwendung neu prüfen; eine Zahl einzutragen erzeugt keinen Port.

5. Bits/s auf **115200** setzen. Bei einer echten UART müssen Pi und USB-UART dazu passen. Die im Screenshot stehenden 19200 sind eine Auswahl, keine Pflichtvorgabe für deinen Sensor.

6. Für alle drei Messwerte Open all input channels wählen. Alternativ über Channel/Pos. gezielt den gewünschten Bereich auswählen, zum Beispiel First = 1 und Last = 3. Die Framewertanzahl im DEFINE-Profil bleibt immer 3.

7. Pi-Sender laufen lassen und Connect klicken. Kontrollieren, dass der richtige Wert auf dem richtigen Kanal ankommt. Zum gezielten Prüfen zunächst nur einen Kanal öffnen oder drei bewusst verschiedene Konstanten senden.

8. Kanalnamen, Einheiten und gegebenenfalls Anzeige-Skalierung passend vergeben. Die physikalische Bedeutung jeder Position schriftlich festhalten. Für den Start kalibrierte Werte auf dem Pi liefern und eine zusätzliche Skalierung in GSVmulti vermeiden.

9. Danach Live-Messung beziehungsweise Recorder starten und eine kurze Aufzeichnung erzeugen. Datei im File Monitor öffnen und Werte sowie Zeitachse kontrollieren. Anschließend die Session speichern; der Hersteller nennt diese Speicherung ausdrücklich auch für die UserDefined-Definition. [1, 4]

Portnummer und Sensoranschluss sind getrennte Einstellungen

COM20 benennt den Transport. Position 1 benennt einen Wert innerhalb des Transports. Derselbe COM-Port kann dadurch drei oder mehr GSVmulti-Kanäle versorgen. Die Einträge im Hauptfenster „Actual Channel“, „COMport No.“ und „Number of Channels“ haben entsprechend unterschiedliche Aufgaben.

Vor dem Verbinden ein Terminal schließen

Ein normaler Hardware-COM-Port kann in der Regel nur von einem Programm gleichzeitig geöffnet werden. Ein Terminal zum Testen deshalb beenden, bevor GSVmulti zugreift. Bei virtuellen Ports hängt die Mehrfachöffnung ausdrücklich vom gewählten Treibertyp ab.

5 Lokaler Anschluss über GPIO und USB

Der einfachste Vorschlag für eine kurze lokale Verbindung verwendet die UART-Signale des Pi und einen USB-UART-Adapter am Windows-PC. Wähle einen Adapter mit **3,3-V-Logikpegeln**. Ein USB-RS232-Kabel mit DB9-Stecker ist elektrisch etwas anderes. Raspberry-Pi-UARTs sind nicht 5-V-tolerant. [5, 6]

```
Raspberry Pi                          Windows-PC
GPIO14 TX ----> RXD am USB-UART ------> USB -> COMx
GPIO15 RX <---- TXD am USB-UART          optional
GND       ---- GND am USB-UART

Pi-Netzteil versorgt den Pi separat. Adapter-VCC bleibt frei.
```

Pi am 40-Pin-Header	USB-UART am PC	Zweck
Pin 8 / GPIO14 / TX	RXD	Pi sendet Daten an den PC.
Pin 10 / GPIO15 / RX	TXD, optional	Rückrichtung; für den reinen Zahlenstrom nicht nötig.
Pin 6 / GND	GND	Gemeinsamer Signalbezug.
3,3 V oder 5 V Versorgung	Nicht verbinden	Bei eigenem Pi-Netzteil keine Versorgung über den Adapter einspeisen.

Die Nummern 8, 10 und 6 sind physische Headerpins; 14 und 15 sind BCM-GPIO-Nummern. Adapterfarben sind nicht einheitlich. Anschlüsse nach Beschriftung oder Datenblatt bestimmen. Auch ein Adapter mit 3,3-V-Signalen kann am VCC-Pin 5 V ausgeben. [6]

Pi konfigurieren

1. In raspi-config unter Interface Options / Serial Port die serielle Login-Shell deaktivieren und die serielle Hardware aktivieren. Menübezeichnungen können je nach Raspberry Pi OS abweichen. Danach neu starten. [5]

2. Den tatsächlich zu GPIO14/15 gehörenden Gerätenamen prüfen. Bei vielen Pi-Modellen bis Pi 4 ist /dev/serial0 der geeignete Alias. Beim Pi 5 gilt die Sonderbehandlung auf der nächsten Seite.

```
readlink -f /dev/serial0
ls -l /dev/serial* /dev/ttyAMA* /dev/ttyS*
sudo apt install python3-serial
python3 gsv_stream_demo.py --serial /dev/serial0 --hz 10
```

„Keine Datei gefunden“ bei einzelnen Glob-Mustern ist möglich und allein noch kein Fehler. Bei Permission denied die Mitgliedschaft in der Gruppe dialout prüfen; nach einer Gruppenänderung neu anmelden. Zuerst die passende UART und ihre Pinbelegung bestätigen.

6 Pi 5 und der Aufbau mit zwei USB Adaptern

Beim Pi 5 den Header UART explizit auswählen

Beim Pi 5 kann /dev/serial0 auf /dev/ttyAMA10 am separaten Debug-Anschluss zeigen. Dann erreicht das Pi-Programm mit diesem Alias nicht den erwarteten TX-Pin am 40-Pin-Header. Die offizielle Dokumentation unterscheidet diese Zuordnungen und nennt eigene Pi-5-Overlays. [5]

1. Auf dem Pi die zur installierten Firmware passende Overlay-Hilfe lesen. Für UART0 am Pi 5 wird das Overlay uart0-pi5 verwendet; lokale Hilfe und /boot/firmware/overlays/README sind die maßgebliche Zuordnung.

```
dtoverlay -h uart0-pi5
pinctrl get 14 15
ls -l /dev/ttyAMA* /dev/serial*
```

2. Wenn dieser UART noch nicht aktiviert ist, das passende Overlay in /boot/firmware/config.txt eintragen. Beispiel nach Prüfung der lokalen Hilfe: dtoverlay=uart0-pi5. Neustarten und die Pin-Funktion sowie den entstandenen Gerätedateinamen erneut prüfen. Keine Debug- oder Login-Ausgaben auf dem Mess-UART zulassen.

3. Den Sender auf genau diesem Gerät starten. /dev/ttyAMA0 ist ein typischer Kandidat für UART0; die Zuordnung muss am konkreten System stimmen. Keinen fest angenommenen Gerätenamen übernehmen, wenn die Beobachtung abweicht.

Die Anleitung verändert deinen Pi nicht. Pi-Modell, Betriebssystem und aktuelle Overlays wurden nicht ausgelesen; die Befehle sind daher Schritte für die Inbetriebnahme vor Ort.

Alternative ohne Pi GPIO und ohne USB Gadget

Du kannst an **beide** Rechner je einen USB-UART-Adapter anschließen. Der Pi und der PC sind dabei jeweils USB-Host. Zwischen den Adaptern liegt eine UART-Verbindung mit gekreuztem TX/RX und gemeinsamem GND. Die Signalpegel beider Adapter müssen zueinander passen, zum Beispiel jeweils 3,3 V.

```
Pi USB-Host -> USB-UART A
                  TXD A ----> RXD B
                  RXD A <---- TXD B   optional
                  GND A ----- GND B
                            USB-UART B -> Windows USB -> COMx
```

Der Pi sendet dann beispielsweise über /dev/ttyUSB0. Der PC öffnet den COM-Port von Adapter B. Beide seriellen Seiten verwenden 115200, 8N1 und zunächst kein Hardware- oder Software-Handshake. Verifiziere den Pi-Gerätenamen, da die Nummer beim Einstecken variieren kann.

Dieser Weg benötigt zwei Adapter, umgeht dafür die GPIO-UART-Zuordnung und die USB-Device-Konfiguration des Pi. Zwei normale USB-Host-Anschlüsse über ein gewöhnliches USB-Kabel zu verbinden erzeugt von allein keine serielle Datenverbindung.

7 Ein USB Kabel mit einem seriellen Gadget

Ein USB-Gadget lässt den Pi gegenüber dem PC als USB-Gerät auftreten. Bei einer seriellen ACM-Funktion schreibt das Pi-Programm auf /dev/ttyGS0; der Windows-Host bekommt einen COM-Port. Linux dokumentiert dafür das Modul g_serial und die ACM-Funktion. [7]

```
Pi-Messprogramm -> /dev/ttyGS0 -> USB ACM
                -> Windows USB-Seriell-Treiber -> COMx -> GSVmulti
```

Voraussetzungen zuerst prüfen

Das konkrete Pi-Modell, der verwendete USB-Anschluss, das Kabel, die Stromversorgung und das Betriebssystem müssen den USB-Device-Betrieb erlauben. Nutze einen Datenport und ein Datenkabel. Der gleiche Steckertyp allein sagt nichts über den verfügbaren USB-Modus aus.

Bei Zero-Modellen ist der OTG-Datenanschluss zu verwenden. Für Pi 4, Pi 5 und Compute Modules nicht ungeprüft ältere Zero-Anleitungen übernehmen. Das aktuelle offizielle Raspberry-Pi-Projekt rpi-usb-gadget führt inzwischen auch neuere Modelle auf, ist aber standardmäßig ein **USB-Ethernet-Gadget**. Es erzeugt ausdrücklich nicht automatisch /dev/ttyGS0. [8]

Beispiel für einen geeigneten Zero Aufbau

Nach Prüfung von Modell und OS kann auf einem dafür freien USB-Device-Controller das folgende Prinzip verwendet werden. Bestehende Gadget-Konfigurationen nicht parallel mit g_serial betreiben: Ein Controller wird von einer Gadget-Konfiguration belegt. Bei einem bereits aktiven Ethernet-Gadget ist eine bewusst konfigurierte Composite-Lösung oder ein anderer Transport erforderlich.

```
# /boot/firmware/config.txt, bei aelteren OS /boot/config.txt
dtoverlay=dwc2,dr_mode=peripheral

# Nach Neustart, wenn kein anderes Gadget den Controller belegt
sudo modprobe g_serial
ls -l /sys/class/udc /dev/ttyGS0
python3 gsv_stream_demo.py --serial /dev/ttyGS0 --hz 10
```

Diese Folge ist eine Inbetriebnahmevorlage, kein auf deinem Pi getestetes Installationsrezept. Erst wenn /dev/ttyGS0 und ein passender Windows-COM-Port vorhanden sind, ist der Transport bereit. An diesem Port keine Login-Konsole betreiben. Die aktuelle Windows-Treiberzuordnung muss ebenfalls geprüft werden.

USB Ethernet bleibt eine IP Verbindung

Falls dein Pi bereits über USB als Netzwerkadapter erreichbar ist, kannst du den TCP-Aufbau der folgenden Seiten darüber verwenden. Ein USB-Netzwerkadapter wird für GSVmulti UserDefined nicht allein dadurch zum COM-Port. Die TCP-COM-Brücke auf Windows bleibt in diesem Fall nötig.

8 Remote Verbindung über TCP und einen COM Port

Für einen selbst geschriebenen Pi-Sender empfiehlt sich ein einfacher Raw-TCP-Dienst. Auf Windows setzt ein virtueller COM-Treiber mit TCP-Client den empfangenen Bytefluss auf einen lokalen seriellen Endpunkt um. ME dokumentiert COM-Mapping als Netzwerkweg für seine seriellen Geräteserver. Der hier vorgeschlagene Pi-Aufbau überträgt dasselbe Prinzip auf eigenen Messwerttext. [2]

```
Pi: 192.168.1.50:5000        Windows
TCP-Server                 TCP-Client der Bruecke
   Messwertframes --------> virtueller COM20 -> GSVmulti

TCP-Port 5000 = Netzwerkdienst    COM20 = Windows-Endpunkt
Beide Nummern sind frei gewaehlte Beispiele.
```

Eine konkrete Möglichkeit mit VSPE

Die Herstellerdokumentation von VSPE beschreibt den Virtual Connector als gemeinsamen virtuellen COM-Endpunkt für zwei Teilnehmer. Der TcpClient erzeugt selbst keinen COM-Port; er verbindet einen vorhandenen Endpunkt mit einem TCP-Dienst. Daraus ergibt sich dieser Aufbau. [9, 10]

1. Eine zum Windows-System passende VSPE-Installation und Lizenz bereitstellen. Im Rahmen dieser Recherche wurde kein Treiber installiert. Einen freien Port, zum Beispiel COM20, für einen **Virtual Connector** anlegen.

2. Einen **TcpClient** hinzufügen. Source COM port = COM20, Remote TCP host = IP oder auflösbarer Hostname des Pi, TCP data port = 5000. Beide Teilnehmer verwenden denselben Connector: TcpClient und GSVmulti.

3. Den zusätzlichen TCP control port für diesen einfachen Python-Server deaktiviert lassen. Keine RFC2217-/Telnet-Verhandlung, keine Kompression und keine Textumwandlung aktivieren. Der Server liefert ausschließlich rohe Messwertbytes.

4. Als Startwert eine Wiederverbindungsverzögerung von 2 Sekunden wählen. Optional Puffer beim Öffnen beziehungsweise Verbinden verwerfen, damit nach einem Ausfall keine alten Werte nachlaufen. Die passenden Optionen hängen vom verwendeten VSPE-Gerätetyp ab.

5. Den Pi-Server starten, danach die VSPE-Konfiguration starten. COM20 in GSVmulti als UserDefined mit dem bekannten DEFINE-Profil öffnen. Bei Connector-Kompatibilitätsproblemen ist Virtual Pair eine dokumentierte Alternative. [10]

```
python3 gsv_stream_demo.py --listen 0.0.0.0 --port 5000 --hz 10
```

0.0.0.0 bedeutet: auf allen IPv4-Schnittstellen lauschen. Für einen SSH-Tunnel stattdessen 127.0.0.1 verwenden. Eine feste DHCP-Zuordnung oder ein stabiler Hostname erleichtert den Betrieb. Der Windows-PC baut die Verbindung zum Pi auf.

9 TCP COM Varianten und die Grenze von NetGSV

Raw TCP und RFC2217 auseinanderhalten

Raw TCP transportiert nur Bytes. RFC2217 ergänzt über Telnet unter anderem Einstellungen und Steuersignale einer seriellen Schnittstelle. Beide Enden müssen dasselbe Verfahren sprechen. Der beiliegende Pi-Server und tcp_to_com.py verwenden ausschließlich Raw TCP. Ein RFC2217-Client kann damit nicht korrekt verhandeln. [11]

Eine Python-Bibliothek, die socket://host:port unterstützt, legt dadurch noch keinen Windows-COM-Port an. Diese URL ist eine Bibliotheksfunktion für das jeweilige Python-Programm. GSVmulti erhält erst durch einen VCOM-Treiber einen passenden Endpunkt.

Alternative mit einem virtuellen COM Portpaar

```
Pi TCP -> Python-Bridge auf Windows -> COM21
                                       | virtuelles Portpaar
                          GSVmulti <- COM20
```

Für diesen Aufbau zuerst mit einem geeigneten signierten Treiber ein Paar COM20 <-> COM21 anlegen. GSVmulti öffnet COM20, die beiliegende Bridge ausschließlich COM21. Zwei unabhängige COM-Ports ohne Paarverbindung reichen nicht. Diese Variante ersetzt den VSPE-TcpClient; beide TCP-Brücken nicht gleichzeitig auf dieselbe Quelle schalten.

```
py -m pip install -r requirements.txt
py tcp_to_com.py --host 192.168.1.50 --port 5000 --serial COM21
```

Die Bridge puffert bis CRLF, prüft Präfix und drei endliche Zahlen und schreibt nur vollständige Zeilen. Nach jeder TCP-Verbindung verwirft sie den ersten Datensatz zur Synchronisation. Bei Netzwerkabbruch verbindet sie sich erneut. Ein Ausfall des COM-Geräts erfordert das Beheben des Ports und einen Neustart der Bridge.

NetGSV ab Version 2 03

Die offizielle Versionshistorie nennt ab 2.03 NetGSV: GSV-6/GSV-8 an einem **ME-proprietären Device Server**, konfiguriert über Options -> Hardware. Nach Neustart werden dort virtuelle COM-Nummern ab 240 verwendet. [12]

Das ist kein belegter allgemeiner Raw-TCP-Import für UserDefined. Ein selbst gebauter Pi-Server ist deshalb nicht automatisch kompatibel. Deine gezeigte Version 2.01 besitzt diese später hinzugefügte Funktion ohnehin nicht. Ein Upgrade allein ersetzt keinen Nachweis der Serverprotokoll-Kompatibilität.

Fernzugriff außerhalb des lokalen Netzes

Raw TCP im Beispiel hat keine Authentifizierung und keine Verschlüsselung. Für Zugriff über fremde Netze einen VPN- oder SSH-Tunnel verwenden. Beispiel: Pi-Server an 127.0.0.1 binden, auf Windows folgenden Tunnel aufbauen und die COM-Brücke auf 127.0.0.1:5000 einstellen:

```
ssh -N -L 5000:127.0.0.1:5000 benutzer@pi-host
```

10 Weitere Anschlüsse und ihre Bedeutung

Schnittstelle	Für den eigenen Pi-Datenstrom	Erforderliche Umsetzung
USB-UART mit 3,3-V-TTL	Direkter Vorschlag über Pi-GPIO oder zweiten Adapter	Windows-Treiber erzeugt COM; UART-Parameter müssen passen.
USB-ACM-Gadget	Möglich bei passendem Device-Modus	Pi-Gadget konfigurieren; Windows muss COM anbieten.
Ethernet / WLAN	Vorgeschlagener Remote-Weg	TCP-Sender plus Windows-TCP-COM-Abbildung.
USB-Ethernet-Gadget	Wie eine andere IP-Verbindung	TCP und Windows-COM-Brücke bleiben erforderlich.
RS232 / RS422	Technisch mit passender Hardware wandelbar	Passende Pegelwandler beziehungsweise USB-Adapter. Kein direkter Anschluss an Pi-GPIO.
Bluetooth	Nur bei einem seriellen, kompatiblen Profil	Ein serialisierender Adapter kann COM liefern. BLE allein ist kein allgemeiner COM-Datenstrom.
CAN / CANopen	Kein Drop-in-Ersatz für UserDefined	Geräte-, Adapter- und Protokollunterstützung erforderlich; keine Annahme beliebiger CAN-Sensoren.
EtherCAT	Kein allgemeiner TCP-Port	Separater Gerätepfad mit unterstützter Hardware und zusätzlicher Master-Software/Lizenz ab 2.03. [12]
I2C / SPI / analog / GPIO	Sensorseite des Pi	Pi liest den Bus beziehungsweise ADC und wandelt Werte in das Textprofil.

Wichtig für die Auswahl eines Sensors

Ein analoger Sensor benötigt eine geeignete Messschaltung und in der Regel einen ADC; ein digitaler Sensor einen passenden Treiber. Ein DMS-Brückensignal lässt sich nicht mit einer seriellen Textdefinition direkt messen. Zuerst entsteht auf dem Pi ein gültiger, kalibrierter Zahlenwert; erst dann wird er übertragen.

Die Eignung einer Verbindung hängt somit an zwei unabhängigen Fragen: Kann dein Pi den Sensor korrekt erfassen? Und kann Windows dem GSVmulti-Parser den daraus erzeugten Text über einen COM-Endpunkt liefern? Diese Trennung erlaubt auch Sensorwechsel, ohne den Transport neu zu entwickeln.

11 Die Beispielprogramme verwenden

Das ZIP-Paket enthält gsv_stream_demo.py, tcp_to_com.py, requirements.txt und README.txt. Alle ausgegebenen Zahlen sind **Demodaten**. Die Programme lesen keinen real angeschlossenen Sensor und emulieren kein natives GSV-Gerät.

Serieller Sender auf dem Pi

```
sudo apt install python3-serial
python3 gsv_stream_demo.py --serial /dev/serial0 --baud 115200

# Alternative bei einem Pi-seitigen USB-UART
python3 gsv_stream_demo.py --serial /dev/ttyUSB0 --baud 115200

# Alternative nach erfolgreicher Gadget-Konfiguration
python3 gsv_stream_demo.py --serial /dev/ttyGS0
```

Die tatsächliche Gerätedatei ist vor dem Start zu prüfen. Der Sender setzt 8N1 und deaktiviert Software- und Hardware-Handshake. Der Windows-Port wird auf dieselbe Bitrate gesetzt. Beim Serialmodus beendet sich das Beispiel bei einem Portausfall; ein späterer Dauerbetrieb benötigt eine überwachte Neustartstrategie.

TCP Sender und ein lokaler Formatcheck

```
python3 gsv_stream_demo.py --listen 0.0.0.0 --port 5000 --hz 10
python3 gsv_stream_demo.py --stdout --count 3
```

Der TCP-Modus wartet auf einen Client und sendet dann selbstständig. Er bedient einen Empfänger zur Zeit und nimmt nach einer Trennung wieder eine Verbindung an. Debugmeldungen gehen auf stderr. --stdout liefert den Messstrom auf stdout und kann ohne serielle Hardware zur Byteprüfung verwendet werden.

Echte Sensorwerte einsetzen

Ersetze im Sender die Funktion read_values(t). Sie muss ein Tupel aus genau drei endlichen Zahlen liefern, zum Beispiel Kraft in N, Weg in mm und Temperatur in °C. Die Reihenfolge ist verbindlich. Sensorbus und Kalibrierung sind dabei vollständig von der gewählten TCP- oder UART-Ausgabe getrennt.

```
def read_values(t):
    force_n = read_force_sensor()    # eigener Treiber
    travel_mm = read_travel_sensor() # eigener Treiber
    temp_c = read_temperature()      # eigener Treiber
    return force_n, travel_mm, temp_c
```

Diese drei read_...-Funktionen sind bewusst Platzhalter. Für eine ausführbare Sensorauslesung werden Sensortyp, Anschluss, Treiber und Kalibrierung benötigt. Der mitgelieferte Sender ist bis dahin durch die vorhandene Demofunktion lauffähig.

12 Datenrate Zeitbasis und Fehlerwerte

Datenmenge vor der Wahl der Baudrate berechnen

Das Beispielprofil verwendet 1 Präfixbyte, 3 Zahlenfelder mit je 10 Bytes, 2 Semikolons und 2 Abschlussbytes: **35 Bytes pro Frame**. Bei UART mit 8N1 benötigt jedes Byte einschließlich Start- und Stopbit 10 Bit.

```
benoetigte Bitrate = Framebytes x Frames/s x 10

10 Hz:  35 x 10  x 10 =  3.500 Bit/s
100 Hz: 35 x 100 x 10 = 35.000 Bit/s
```

19200 Bit/s genügt für das 10-Hz-Beispiel mit Reserve. Für 100 Hz genügt es nicht. 115200 Bit/s benötigt bei 100 Hz rund 30,4 Prozent der nominalen UART-Kapazität. PC-Verarbeitung, Treiber und Sensorauslesung können die praktisch erreichbare Rate zusätzlich begrenzen.

Eine höhere COM-Baudrate erhöht nicht die Abtastrate des Sensors. Bei Raw TCP ohne physische UART ist die eingestellte Baudrate zunächst eine Treiber-/Anwendungsgröße; einzelne virtuelle Treiber können sie aber als Drosselung emulieren. Bei einer echten UART ist sie die physische Übertragungsrate.

Messzeit und Empfangszeit getrennt prüfen

GSVmulti sieht beim einfachen Textprofil Zahlen in einer Reihenfolge. Ein zusätzlich gesendeter Zeitstempel wäre zunächst ein weiterer Zahlenwert und wird nicht automatisch zur Zeitachse. Netzwerkpufferung kann die Ankunft verzögern oder mehrere Frames gebündelt liefern. Die tatsächliche Zeitbasis der UserDefined-Aufzeichnung muss deshalb bei der Abnahme geprüft werden.

Starte mit einer festen Senderate von 10 Hz. Sende 600 Frames in ungefähr einer Minute und prüfe Anzahl und aufgezeichnete Dauer. Wiederhole den Versuch mit einer kontrollierten Unterbrechung. Eine Änderung an der GSVmulti-Schaltfläche Data Frequency ist kein Beleg dafür, dass dein eigenes Pi-Programm umkonfiguriert wurde.

Das Beispiel verwendet monotonic für die Taktung und holt nach langen Pausen keine verpassten Messschritte im Eiltempo nach. Für präzise synchronisierte Messungen sind eine definierte Erfassungstaktung und eine gesonderte Zeitkonzeption nötig.

Fehlende oder ungültige Messwerte

Das Beispiel lehnt NaN, Inf und Werte außerhalb seines festen Darstellungsbereichs ab und stoppt. Es setzt Messfehler nicht stillschweigend auf 0. Für den Produktivbetrieb musst du festlegen, ob ein kompletter Frame ausfällt, eine separate Gültigkeitsgröße mitgesendet oder die Messung beendet wird. Die NaN-Behandlung von GSVmulti ist hier nicht bestätigt.

Ein konstanter zusätzlicher Statuskanal wäre ein eigenes neues Profil, zum Beispiel vier statt drei Zahlen. Die Bedeutung von Status und Datenlücken sowie ihre Wirkung auf die Zeitachse muss dann ebenfalls getestet werden.

13 Verbindung prüfen und Fehler eingrenzen

Beobachtung	Gezielt prüfen
Kein Windows-COM-Port	USB-Datenkabel, Adaptertreiber, Gerätemanager; bei TCP zuerst VCOM-Gerät anlegen.
COM122 ist gewählt, aber keine Daten	Existiert dieser Port tatsächlich? Der Screenshotwert allein weist keine Verbindung nach.
Port kann nicht geöffnet werden	Terminal oder andere Anwendung schließen; richtigen Endpunkt des virtuellen Paars verwenden.
Unlesbare Zeichen	UART-Baudrate, 8N1, GND und Logikpegel prüfen. Keine Debug-Konsole auf dem Datenport.
Text lesbar, GSVmulti ohne Werte	CRLF als Bytes prüfen, Präfix %, Anzahl 3 und Semikolon kontrollieren.
Werte vertauscht	Kanalposition und Pi-Feldreihenfolge vergleichen; drei verschiedene Konstanten senden.
TCP erreichbar, GSVmulti ohne Daten	Raw TCP gegen RFC2217 prüfen; VCOM-Mapping und Connector/Pair-Endpunkte kontrollieren.
Nach Ausfall alte Werte	TCP-/COM-Puffer und Replay prüfen. Auf neue vollständige Frames synchronisieren.
Zeitachse unplausibel	Frames pro Sekunde und Aufzeichnungsdauer messen; Bursts und Datenlücken auswerten.
Pi 5 sendet scheinbar ins Leere	/dev/serial0 könnte den Debug-UART adressieren; GPIO-Funktion und konkreten UART prüfen.

Hilfreiche Windows Befehle

```
py -m serial.tools.list_ports -v
py -m serial.tools.miniterm COM20 115200
Test-NetConnection 192.168.1.50 -Port 5000
```

miniterm erst verwenden, wenn GSVmulti den Port geschlossen hat. Nach dem Test das Terminal wieder schließen. Test-NetConnection bestätigt nur die TCP-Erreichbarkeit; es beweist weder das Textformat noch die vollständige Datenkette.

Wiederaufnahme nicht voraussetzen

Die Herstelleranleitung beschreibt die automatische Wiederverbindung mit gerätespezifischen Lese- und Start-Kommandos ausdrücklich für native GSV-Verstärker. Daraus folgt keine garantierte Wiederanlaufsequenz für UserDefined. VCOM-Brücke und Sender benötigen deshalb eine eigene Strategie; nach einem Abbruch muss auch die Weiteraufzeichnung geprüft werden. [1, Seite 122]

14 Abnahme am tatsächlichen Aufbau

Diese Abnahme ist der noch ausstehende Nachweis, dass deine konkrete Kombination aus Pi, Treiber und GSVmulti 2.01 Pro funktioniert. Die Prüfungen lassen sich zunächst mit Demodaten durchführen, bevor ein realer Sensor angeschlossen wird.

1. Transport sichtbar: COM-Port im Windows-Gerätemanager dokumentieren. Bei UART den Pi-Geräteknoten und die Verdrahtung festhalten; bei TCP die Pi-IP, TCP-Port und Windows-VCOM-Konfiguration.

2. Byteformat korrekt: Beispieldatensatz an einer Stelle der Kette erfassen. Beginn 25 hex für %, Ende 0D 0A hex, drei Zahlen, Dezimalpunkt und Semikolon bestätigen. Für das feste Profil sind es 35 Bytes pro Frame.

3. Drei Kanäle korrekt: Mit drei klar unterschiedlichen konstanten Werten testen. GSVmulti-Positionen, Vorzeichen und Größenordnung prüfen. Danach langsam veränderliche Demodaten verwenden.

4. Rate und Aufzeichnung: Eine Minute mit 10 Hz aufzeichnen. Rund 600 gesendete Frames mit der Anzahl gespeicherter Datensätze beziehungsweise Werte pro Kanal und der Zeitachse vergleichen. Start-/Stoppränder berücksichtigen. Keine ungeprüfte Gleichsetzung von TCP-Paketen und Frames.

5. Unterbrechung: Netzwerk beziehungsweise Datenleitung kontrolliert trennen und wieder verbinden. Prüfen, ob neue Werte weiterlaufen, ob ein manuelles Neuöffnen nötig ist und wie die Aufzeichnung die Lücke abbildet. Kein unbemerktes Nachliefern alter Werte zulassen.

6. Realer Sensor: Erst jetzt read_values ersetzen. Nullpunkt, bekannte Referenzwerte, Einheit und Kalibrierung gegen ein unabhängiges Referenzmittel prüfen. Danach einen längeren Lauf mit der vorgesehenen Datenrate durchführen.

7. Wiederherstellung: Session speichern, Anwendung neu starten, Session laden und dieselben Schritte wiederholen. Bei Netzwerkbetrieb zusätzlich den Start der VCOM-Brücke und ihre gespeicherte Konfiguration prüfen.

Was du für einen konkreten Sensor noch festlegen musst

Angabe	Warum sie gebraucht wird
Pi-Modell und Raspberry Pi OS	UART-Zuordnung oder geeigneten USB-Device-Modus bestimmen.
Sensortyp und Schnittstelle	Treiber, ADC und elektrische Beschaltung bestimmen.
Einheiten und Kalibrierung	Rohwerte in physikalisch sinnvolle Zahlen umrechnen.
Kanalzahl und Sollrate	Frameprofil, Übertragungsreserve und Messzeitbasis festlegen.
Lokal oder remote	Kabelaufbau beziehungsweise VCOM und geschützten Netzwerkweg wählen.

15 Quellen und Nachweisgrenzen

Primärquellen wurden am 28.10.2026 geprüft. Die im Web verfügbare Bedienungsanleitung v2.6 ist ein Dokumentstand und nicht die Versionsnummer deiner installierten Anwendung. Die Herstellerseite nennt zum Recherchezeitpunkt GSVmulti 2.03 als Download. Ein Update wurde nicht vorgenommen. [4, 12]

[1] ME Bedienungsanleitung GSVmulti v2.6
Insbesondere S. 118-121 UserDefined; S. 122 Wiederaufnahme; S. 105-115 Interfaces und Frames.

[2] ME GSV 8 Netzwerk Schnittstelle
Herstelleraufbau mit Geräteserver und erforderlichem COM-Mapping.

[3] ME Einstellung des Datenformats
Native GSV-Datenformate und Übertragungsverhalten.

[4] ME Produktseite GSVmulti
Funktionsumfang, TDMS, Export und aktuelle Downloads.

[5] Raspberry Pi Configuration UART
3,3-V-Pegel, serielle Konsole, Modellunterschiede und Linux-Gerätenamen.

[6] FTDI TTL 232R 3V3
Beispiel eines USB-UART mit 3,3-V-Signalen und separatem 5-V-VCC.

[7] Linux Kernel Gadget Serial Driver
g_serial, ACM und /dev/ttyGS0. Ältere Grundlagenquelle; aktuelle Treiberzuordnung vor Ort prüfen.

[8] Raspberry Pi rpi usb gadget
Aktuelles offizielles Projekt für USB-Ethernet; standardmäßig kein serielles Gadget.

[9] Eterlogic VSPE TCP Client
TCP-Datenport, optionaler Control-Port und Wiederverbindung.

[10] Eterlogic VSPE Virtual Connector
Virtueller Port für zwei Teilnehmer; Virtual Pair als Alternative.

[11] pySerial URL Handlers
Unterschied zwischen socket und RFC2217; Bibliotheks-URL statt Windows-VCOM.

[12] ME Versionshistorie GSVmulti
Version 2.03 mit NetGSV und EtherCAT; Version 2.01 vom November 2023.

Lokal bestätigt und noch offen

Die beiden Nutzeranhänge belegen die sichtbaren Felder der 2.01-Pro-Oberfläche. Der laufende Installationspfad wurde lokal gelesen. In der Windows-SERIALCOMM-Abfrage war beim Prüfen COM1 sichtbar; ein funktionsfähiger COM122, COM20 oder COM21 wurde nicht nachgewiesen. Es wurden keine existierenden Messungen oder Session-Dateien geändert.

Eigene Entwürfe sind das %-Dreikanalprofil, die Verdrahtungsvarianten, der TCP-Python-Server und die alternative Bridge. Die Softwarebeispiele wurden separat auf ihre Datenstromfunktionen geprüft. Kompatibilität mit der konkreten GSVmulti-Instanz und echter Hardware bleibt bis zur Abnahme auf der vorigen Seite offen.